Practical Exam: SWD392 –SP26

Duration: 85 Minutes

Instructions:

- Internet access is NOT permitted during the exam.
- You are allowed to use UML modeling tools in offline mode *(Visual Paradigm, Visio, Draw.io)*

Submission Requirements

- Submit your answer as a single Word file or a .rar/.zip archive.
- Name your file exactly:

  SWD392_PE_SPR26_[StudentID].docx or
  SWD392_PE_SPR26_[StudentID].rar

Submissions with incorrect file names may not be graded.

Business Overview:

StayEase is a comprehensive digital platform designed to connect **Property Owners (Hosts)** with **Travelers (Customers)**. The system streamlines the entire process of finding, reserving, and managing short-term accommodations.

- **Customers:** Can search for available rooms based on location and dates, create reservations, and process payments. They are responsible for paying a deposit to confirm their booking.
- **Hosts:** Responsible for listing their properties, managing room details (amenities, pricing), and performing room inspections upon guest departure.
- **Staff/Administrators:** Oversee the platform's operations, manage user accounts, and generate financial reports.

The purpose of this exam is to assess your ability to design software architecture and UML models rather than implement program logic

This examination evaluates students' understanding of software architecture and UML-based system design. Students are **not required to write any source code**. All answers must be presented using UML diagrams.

Students may make reasonable assumptions where necessary, as long as those assumptions do not contradict the given business context.

Question 1 (3.0 points) – Design-level Class Diagram

Business Scenario: You are required to design a **design-level Class Diagram** for the **StayEase** homestay booking system. The system is built around a predefined set of classes representing users, rooms, reservations, and payments. Your task is to enrich these classes with appropriate attributes, methods, and relationships to reflect the business logic.

In this system, different types of users exist with shared and specialized responsibilities. **Customers** can browse and create reservations, each **Reservation** is associated with a specific **Room**, and **Payments** are recorded for each reservation to track deposits and final balances. The system follows the **Repository Pattern** to separate business logic from data access logic, ensuring the service layer remains independent of the database implementation.

Your Class Diagram must satisfy the following requirements:

- **Inheritance:** The classes **Customer** and **Host** (Property Owner) are specialized types of **User** and must inherit from it.
- **Multiplicity & Association:** A customer can create multiple reservations, while each reservation is linked to exactly one room.
- **Composition:** A reservation involves one or more payments (such as a 'Deposit' or 'Final Payment'). A payment record cannot exist without a corresponding reservation.
- **Layered Dependency:** Business logic related to room availability and booking is handled by a service class (**BookingService**). This service class depends on repository interfaces to interact with data.
- **Interface Implementation:** All repository classes (e.g., RoomRepository, ReservationRepository) must implement a generic interface named **IGenericRepository**.

Your diagram must correctly apply UML relationships, including:

- Inheritance (Generalization) and Interface Implementation (Realization).
- Association, Aggregation, and Composition where appropriate.
- Dependency relationships between the Service layer and the Repository layer.
- **Multiplicity, attributes, and methods** should be defined clearly and reasonably for each class (e.g., status for Rooms, totalAmount for Reservations).

Question 2 (4.0 points) – Sequence Diagram

Business Scenario: A **Customer** wants to book a homestay room through the **StayEase** platform. The process starts when the customer interacts with the **BookingUI** (a boundary object) to submit their booking request (Room ID and stay dates).

The request is handled by a **BookingController** (a control object). The system then performs the following steps:

1. The controller calls a **RoomRepository** to verify the current status and availability of the selected room for the given dates.
2. If the room is available, the controller proceeds to call a **PaymentService** to process a mandatory **Deposit Payment** via an external gateway.
3. Depending on the payment result:
    - **If successful:** The controller creates a new **Reservation** record, saves it using the **ReservationRepository**, and updates the room's status to "Booked".
    - **If failed:** The controller returns an error message to the **BookingUI** and the process stops.

Your Sequence Diagram must meet the following requirements:

- The diagram must be designed using **Layered Architecture**.
- The interaction flow must clearly show:
    - A user-facing component (**BookingUI**).
    - A controller/service component handling business logic (**BookingController**).
    - Repository components responsible for data access (**RoomRepository**, **ReservationRepository**).
    - Domain entities such as **Reservation** and **Room**.
- **Repository Pattern** usage must be explicit through repository method calls (e.g., getById, save, updateStatus).
- Conditional behavior must be modeled using **alt (alternative) fragments** for:
    1. Room availability (Available vs. Occupied).
    2. Payment status (Success vs. Failure).

Question 3 (3.0 points) – Activity Diagram

Use Case: Check-out and Inspection

Business Scenario: This workflow describes the process when a **Guest** checks out of a homestay. The **Host** (Property Owner) must perform a physical inspection of the room to ensure no damage has occurred.

The process follows these steps:

1. The **Host** starts by inspecting the room condition.
2. The **Host** then enters the inspection results into the **StayEase** system.
3. The **System** evaluates the inspection report:
    - **If damage is found:** The **System** calculates an additional **Damage Fee** and adds it to the **Final Invoice**.
    - **If no damage is found:** The **System** generates the **Final Invoice** based only on the remaining stay balance.
4. The **Host** requests the **Guest** to settle the payment.
5. After the payment is confirmed, the **System** updates the **Reservation** status to "Completed" and changes the **Room** status to "Cleaning".

Task: You are required to model this process using an **Activity Diagram** with the following characteristics:

- The diagram must include at least **two swimlanes** representing **Host** and **System** responsibilities.
- **Decision nodes** must be used to determine whether additional fees (Damage Fees) are applied.
- The workflow must clearly show the update of both **Reservation** and **Room** statuses.
- **Start and end points** of the activity must be clearly indicated using proper UML notation.
- Your diagram must be created at the design level, including Actors/Roles and Components/Services.

END OF EXAMINATION

(Invigilators will not provide further explanations)